

CAROM: Content-Addressable Recurrent Operator Machine

GPU Mechanism Screen Report — Experiment r1

Research Agent

2026-07-20

Abstract

CAROM is a differentiable register machine that separates control (command tokens route computation via a learned routing table) from data (workspace slots operated on by a shared operator core). A single `OperatorCore` module (K parallel read-transform-gate-write operators) supports three execution semantics: **scheduled** (external clock), **fixed-point** (deep equilibrium with contraction pressure), and **itinerant** (GLV competition dynamics producing heteroclinic channels). This screen tests all four variants on a compositional sequence-transform task with a common operator core, common task, and shared seed. The itinerant-fixed (hand-built chain) variant achieves near-perfect accuracy (99.6%), dramatically outperforming scheduled (77.1%) and fixed-point (56.0%). The free-rho itinerant variant reaches 86.5%, confirming that learned inhibition can produce useful itinerant dynamics without a hand-built chain.

1 Summary

Experiment ID: 20260720T000500Z-carom-gpu-screen-r1

Status: completed (all four arms, exit code 0)

Pod: 00e42nckt4vm1v (A100 SXM4 80GB, Secure Cloud)

Cost bound: USD 10.00

Approval: Ben Goertzel, 2026-07-19

2 Task

The task is a compositional sequence-transform problem over a workspace of $n = 6$ slots in $\mathbb{Z}_8$. Programs consist of up to $L = 5$ commands drawn from a vocabulary of 10 tokens (8 distinct transforms plus 2 synonyms that force routing reuse). Commands include increment, double, negate, reverse, shift, swap, and cumulative-sum. A program is executed in presentation order; shorter programs are padded with `noop`.

The model receives the command sequence C and the initial workspace X , and must predict the final workspace $Y = f_C(X)$.

3 Architecture

3.1 Operator Core

The shared `OperatorCore` implements $K = 16$ parallel operators. Each operator performs:

1. **Read:** Compute attention-weighted read from workspace w using concatenated content and positional features.
2. **Transform:** Two-layer MLP with GELU activation.
3. **Gate:** Sigmoid gate $g = \sigma(W_g z + b_g)$ controls write amplitude.
4. **Write:** Produce update $u = g \cdot \tanh(z)$.

The core parameters are:

- $W_q, W_k, W_v \in \mathbb{R}^{K \times 3d \times d}$ (attention projections)
- $T_1 \in \mathbb{R}^{K \times 4d \times 2d}, T_2 \in \mathbb{R}^{K \times 2d \times d}$ (transform MLP)
- $W_g \in \mathbb{R}^{K \times d \times d}, b_g \in \mathbb{R}^{K \times d}$ (gate)
- LayerNorm on workspace content

Model dimension $d = 64$, $K = 16$ operators, total parameters $\approx 1.32\text{M}$.

3.2 Execution Semantics

Scheduled

External clock iterates for $T = 5$ steps. At step t , routing weights $\beta_t = \text{softmax}(\text{route}(C_t)/\tau)$ select a mixture over operators:

$$w \leftarrow w + \sum_{k=1}^K \beta_{t,k} u_k$$

Fixed-Point (Deep Equilibrium)

Starting from w_0 , iterate the operator mixture for 8 sweeps with relaxation factor $\alpha = 0.5$:

$$w \leftarrow (1 - \alpha) w + \alpha \left(w + \sum_k \beta_k u_k \right)$$

Contraction pressure $\|r_{\text{last}}\|$ is added to the loss. Residuals $\|w_{t+1} - w_t\|$ are logged per sweep.

Itinerant (GLV Competition)

Operators compete via Generalized Lotka–Volterra dynamics over $T_{\text{dyn}} = 70$ steps with step size $\Delta t = 0.1$, fatigue, and leak. The activity vector $a(t) \in \mathbb{R}^K$ evolves as:

$$\dot{a}_k = a_k \left(r_k - \sum_j M_{kj} a_j - \phi_k \right)$$

where M is the inhibition matrix, r_k is the intrinsic growth rate, and ϕ_k is a fatigue term. In the **fixed-chain** variant, M is hand-built to produce a predetermined heteroclinic channel visiting operators in sequence. In the **free-rho** variant, M is learned.

4 Configuration

Parameter	Value
Seed	7
Steps	4000
Batch size	128
Model dimension d	64
Operators K	16
Optimizer	AdamW (lr = 2e-3, wd = 1e-4)
Scheduler	OneCycleLR
Gradient clip	1.0
Fixed-point sweeps	8, $\alpha = 0.5$
Itinerant dynamics steps	70, $\Delta t = 0.1$
Eval batch	512
Eval frequency	every 100 steps
Checkpoints	at steps 0, 1000, 2000, 3000, 3999
GPU	NVIDIA A100-SXM4-80GB

5 Results

5.1 Final Accuracy

Variant	Final Accuracy	Time (s)	Parameters
Scheduled	0.771	82	1,322,472
Fixed-Point	0.560	203	1,322,792
Itinerant-Fixed	0.996	1,403	1,326,925
Itinerant-Free	0.865	1,417	1,326,925

5.2 Learning Curves

Scheduled

```
Step 0: loss=2.1033 acc=0.1416 |g|=0.3321 t=0s
Step 100: loss=2.0524 acc=0.1634 |g|=0.2158 t=3s
Step 500: loss=1.8065 acc=0.2751 |g|=0.3357 t=11s
Step 1000: loss=1.1893 acc=0.5417 |g|=0.7741 t=21s
Step 2000: loss=0.6308 acc=0.7803 |g|=0.2982 t=42s
Step 3000: loss=0.4709 acc=0.8167 |g|=0.1497 t=62s
Step 3999: loss=0.4092 acc=0.7965 |g|=0.2374 t=82s
```

Rapid early learning, plateaus around 0.78–0.82 after step 2000. Loss stabilizes near 0.41 but does not converge.

Fixed-Point

```

Step 0: loss=2.1040 acc=0.1416 |g|=0.3321 t=0s res_last=0.0007
Step 100: loss=2.0598 acc=0.1576 |g|=0.2183 t=5s res_last=0.0012
Step 500: loss=1.9347 acc=0.2161 |g|=0.2256 t=26s res_last=0.0248
Step 1000: loss=1.7429 acc=0.3255 |g|=0.4034 t=51s res_last=0.0413
Step 2000: loss=1.4228 acc=0.4492 |g|=0.3736 t=102s res_last=0.0515
Step 3000: loss=1.0729 acc=0.5999 |g|=0.4891 t=153s res_last=0.0542
Step 3999: loss=0.9462 acc=0.6081 |g|=0.4116 t=203s res_last=0.0541

```

Slowest learning. Convergence is poor despite contraction pressure. Residuals initially near zero, grow to ~ 0.054 and stabilize—indicating the fixed-point iteration does not fully converge within 8 sweeps.

Per-sweep residuals (final evaluation):

```

Sweep 0: 0.100161
Sweep 1: 0.089658
Sweep 2: 0.076235
Sweep 3: 0.066389
Sweep 4: 0.060115
Sweep 5: 0.055893
Sweep 6: 0.052934
Sweep 7: 0.050774

```

Residuals decrease monotonically (validating contraction), but the final residual (0.051) is non-negligible.

Itinerant-Fixed (Hand-Built Chain)

```

Step 0: loss=2.1032 acc=0.1416 |g|=0.3357 t=1s
Step 100: loss=2.0455 acc=0.1592 |g|=0.2204 t=35s
Step 500: loss=1.6875 acc=0.3141 |g|=0.5610 t=180s
Step 1000: loss=0.7018 acc=0.7214 |g|=0.8027 t=353s
Step 2000: loss=0.5029 acc=0.8288 |g|=0.6838 t=701s
Step 3000: loss=0.0333 acc=0.9935 |g|=0.7993 t=1055s
Step 3999: loss=0.0093 acc=0.9977 |g|=0.2981 t=1403s

```

Slow start (each step is $\sim 17\times$ more expensive than scheduled due to 70-step GLV integration), but after step 2000, accuracy climbs steeply to near-perfect. Final loss 0.009 indicates near-convergence.

Final activity mean (5 slots): [0.0595, 0.0611, 0.0520, 0.0500, 0.0518]

Active operators at final step: 0 (all decayed)

Itinerary order (sampled dynamics steps):

```

Step 0: active=[0]
Step 3: active=[0, 1]
Step 6: active=[1]
Step 9: active=[1, 2]
Step 12: active=[2]
Step 15: active=[2, 3]
Step 18: active=[3]
Step 21: active=[]
Steps 24-69: active=[] (all decayed)

```

Clean sequential visitation: operator $0 \rightarrow 0+1 \rightarrow 1 \rightarrow 1+2 \rightarrow 2 \rightarrow 2+3 \rightarrow 3 \rightarrow \text{empty}$. This is the expected heteroclinic channel for the hand-built chain.

Itinerant-Free (Learned Inhibition)

```
Step 0: loss=2.1033 acc=0.1416 |g|=0.3355 t=1s
Step 100: loss=2.0478 acc=0.1644 |g|=0.2181 t=36s
Step 500: loss=1.8504 acc=0.2643 |g|=0.3268 t=175s
Step 1000: loss=1.3220 acc=0.4180 |g|=0.6413 t=353s
Step 2000: loss=0.5489 acc=0.8135 |g|=0.5365 t=707s
Step 3000: loss=0.2261 acc=0.8968 |g|=0.2709 t=1065s
Step 3999: loss=0.1943 acc=0.8802 |g|=0.1942 t=1417s
```

Learns faster than fixed-point but slower than itinerant-fixed. Plateaus around 0.88—good but clearly below the hand-built chain.

Final activity mean (5 slots): [0.0544, 0.0688, 0.0508, 0.0819, 0.9597]

Active operators at final step: 0.053 (near-zero)

The high activity in slot 4 (0.960) suggests one operator remains partially active—a different dynamics regime than the fixed-chain.

Itinerary order (sampled dynamics steps):

```
Step 0: active=[0]
Step 3: active=[0, 1]
Step 6: active=[0, 1]
Step 9: active=[1]
Step 12: active=[1, 2]
Step 15: active=[1, 2]
Step 18: active=[2]
Step 21: active=[2]
Step 24: active=[2]
Step 27: active=[2, 3]
Step 30: active=[2, 3]
Step 33: active=[3]
Step 36: active=[3]
Step 39: active=[3]
Step 42: active=[]
Steps 45-69: active=[] (all decayed)
```

The learned inhibition produces a sequential visitation but with more overlap and dwell time at each operator. The sequence is correct but less crisp than the hand-built chain.

6 Analysis

6.1 Itinerant Dynamics Are Decisive

The itinerant-fixed variant’s near-perfect accuracy (99.6%) vs scheduled (77.1%) demonstrates that sequential operator activation via heteroclinic channels is a powerful execution paradigm for compositional tasks. The key advantage: the itinerant dynamics enforce a temporal ordering on operator application that the scheduled variant must learn implicitly through routing alone.

6.2 Fixed-Point Convergence Is the Bottleneck

The fixed-point variant’s poor performance (56.0%) is explained by incomplete convergence: residuals decrease monotonically across sweeps but stabilize at ~ 0.051 , indicating the equilibrium is

not reached within 8 sweeps. This is consistent with the CPU sandbox finding that residuals grow without contraction pressure. The contraction penalty is necessary but not sufficient.

6.3 Learned Inhibition Works But Needs Refinement

The free-rho variant (86.5%) confirms that learned GLV inhibition can produce useful heteroclinic channels without a hand-built chain. The itinerary is sequentially correct but has more overlap and dwell time, suggesting the inhibition matrix needs stronger off-diagonal terms or longer dynamics steps for crisper transitions.

6.4 Routing Bottleneck

All variants share the same routing table *A* mapping command tokens to operator mixtures. The two synonym tokens (`inc_syn`, `rev_syn`) force the same routing as their primary counterparts—a bottleneck that tests whether the operator core can generalize across shared routing. The itinerant-fixed variant’s near-perfect accuracy suggests this bottleneck is not the limiting factor for compositional generalization.

6.5 Compute Cost

The itinerant variants are $\sim 17\times$ more expensive per step than scheduled (35s/100 steps vs 2s/100 steps) due to 70-step GLV integration. This is the dominant cost. For larger tasks, amortizing the dynamics (e.g., learned step count or adaptive stopping) will be essential.

7 Artifact Provenance

File	SHA-256
summary.json	74b3e0796693fec3...0ff0b7f
scheduled.json	8dbc397adcf96f39...0e85315c
fixedpoint.json	fef5fc7d803d82f3...cd2c731e
itinerant-fixed.json	322d279ad80bc7e9...f8f2d4e52
itinerant-free.json	78d35512b66d54a3...035b321d1
mechanism_screen.log	dd2fefac4dfdad73...68e0fa52

Artifacts stored at: `projects/carom/experiments/20260720T000500Z-carom-gpu-screen-r1/artifacts/carom`

8 Validity Gates

- ✓ Fixed-point residuals decrease monotonically across sweeps (contraction confirmed)
- ✓ Hand-built chain itinerancy is distinct from free-rho itinerancy (different activity patterns, different accuracy)
- ✓ All four variants share common task, operator core, and seed
- ✓ Exit code 0, all artifacts retrieved and verified

9 Limitations

1. **Single seed.** No variance estimate. The ranking is clear but confidence intervals are unknown.
2. **Toy task.** 6 slots, 8 values, 5 commands. Generalization to larger workspaces or longer programs is untested.
3. **No implicit-gradient DEQ.** Fixed-point variant uses explicit sweeps, not backward-through-fixed-point. Performance may differ with implicit gradients.
4. **No hyperparameter search.** All variants use the same lr, batch size, and schedule. Itinerant-specific tuning may improve free-rho.
5. **One-off screen.** Not a broad claim of algorithmic superiority.

10 Conclusions

Finding	Status
Itinerant (fixed-chain) execution achieves near-perfect accuracy on compositional transforms	Observed
Scheduled execution plateaus at $\sim 77\%$ on the same task	Observed
Fixed-point execution is limited by incomplete convergence (residual ~ 0.05)	Observed
Learned (free-rho) GLV inhibition produces a correct but less crisp itinerary	Observed
Temporal ordering via heteroclinic channels is the key mechanism for compositional generalization	Inferred
Stronger inhibition or adaptive dynamics could close the free-rho gap	Hypothesis
The routing bottleneck (synonym tokens) is not the limiting factor	Inferred

11 Next Steps

1. Multi-seed (3–5 seeds) confirmation of the accuracy ranking.
2. Adaptive itinerant dynamics: learned step count or early stopping.
3. Stronger free-rho inhibition: sweep fatigue/leak or initialize from fixed-chain prior.
4. Larger tasks: longer programs, bigger workspace, more operators.
5. Implicit-gradient DEQ training for the fixed-point variant.
6. Scrambled-order discovery: can free-rho learn non-sequential itineraries?

Report generated 2026-07-20 04:50 PDT. Source: experiment r1 artifacts, CAROM repository [projects/carom/repos/carom/](https://github.com/robertmiller/carom).